

Relatorio de Auditoria de Seguranca

Iseltoken

Data: 09/09/2026

Escopo auditado: backend NestJS + frontend React Router 7 + auth Redis + persistencia SQLite/MySQL + integracoes (AWS S3, EFI Bank, AWS SES)

Categorias analisadas: 5 (Isolamento por userId, Autorizacao server-side, IDOR, Segredos hardcoded, XSS/Input nao tratado)

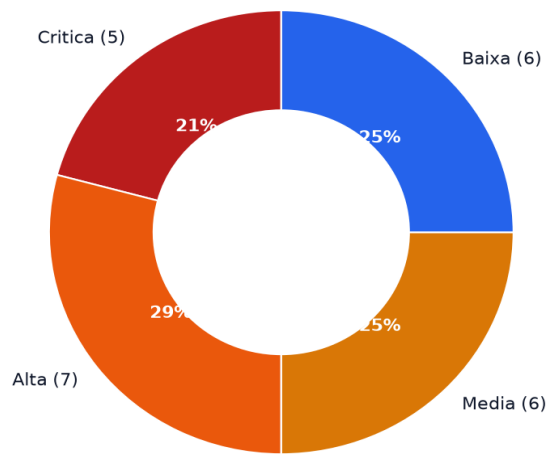
Nota metodologica

Cada categoria foi mapeada para a stack real do projeto. Para **Isolamento (Cat. 1)**: o mecanismo deste projeto eh o filtro manual por `req.user.id` no controller/service Prisma (nao ha RLS, middleware de tenant ou schema-per-tenant). Para **Autorizacao (Cat. 2)**: o controle esta em NestJS Guards (AuthGuard, AdminGuard, ComplianceGuard, FinanceRoleGuard, TwoFactorGuard). Frontend usa `useUserRole()/hasRole()` apenas para esconder UI. Para **IDOR (Cat. 3)**: 100% dos handlers com parametro `:id` foram percorridos. Para **Segredos (Cat. 4)**: `.env*`, `.env.example`, defaults em codigo e historico git. Para **XSS (Cat. 5)**: `dangerouslySetInnerHTML`, `v-html` e templates de email com interpolacao direta.

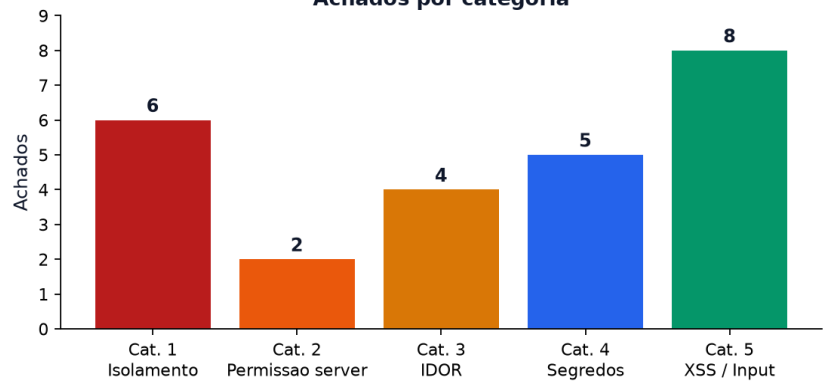
Resumo executivo

Foram encontrados **25 achados** no escopo auditado, distribuidos em **5 criticos**, **7 altos**, **6 medios** e **6 baixos**.

Achados por severidade



Achados por categoria



Mapa visual das issues

Cada linha corresponde a uma issue no arquivo issues.json. A coluna de severidade usa a mesma cor aplicada nos detalhes técnicos abaixo.

ID	Severidade	Prioridad e	Issue
F-02	CRÍTICA	P1	PUT /campaigns/:id/resources sem autenticacao nem checagem de owner
F-03	ALTA	P1	GET /startup/:id publico retorna email do founder (LGPD)
F-04	MÉDIA	P2	GET /campaigns e /campaigns/:id publicos sem filtro de status
F-05	MÉDIA	P2	GET /campaigns/:id/resources sem filtro de status e projecao segura
F-06	BAIXA	P3	POST /payment aceita subscriptionId/investmentId/campaignId do body sem ownership
F-07	BAIXA	P3	GET /admin/config/fundraising e /admin/config/parameters acessiveis a qualquer user autenticado
F-08	MÉDIA	P2	AdminGuard usado sem AuthGuard — rotas admin inacessiveis OU 403 fixo
F-09	BAIXA	P3	Frontend tem useUserRole()/hasRole() mas o backend eh a fonte de verdade — sem furo direto encontrado
F-10	CRÍTICA	P1	GET /tokens/:id — IDOR permite ler qualquer token pelo UUID
F-11	CRÍTICA	P1	PUT /campaigns/:id/resources — IDOR permite alterar recursos de qualquer campanha
F-12	ALTA	P1	GET /startup/marketplace/private/:slug — sem checar se user eh founder/compliance/admin
F-13	MÉDIA	P2	GET /campaigns/:id — projeção pública expunha dados completos da campanha

ID	Severidade	Prioridade	Issue
F-14	CRÍTICA	P1	JWT_SECRET sem fallback e validado no bootstrap
F-15	ALTA	P1	WEBHOOK_HASH_SECRET obrigatório e sem fallback hardcoded
F-16	ALTA	P1	Credenciais operacionais exigem rotação e secret manager
F-17	MÉDIA	P2	EFI_WEBHOOK_HMAC_SECRET depende de rotação operacional
F-18	BAIXA	P3	VITE_EFI_PAYEE_CODE público por design (informativo)
F-19	CRÍTICA	P1	Templates de email escapam dados e validam links dinâmicos
F-20	ALTA	P1	Templates welcome e forgot-password escapam dados do usuário
F-21	ALTA	P1	EmailRenderService usa escaping contextual no HTML
F-22	ALTA	P1	EmailService escapa texto simples antes de convertê-lo em HTML
F-23	MÉDIA	P2	Previews HTML de templates usam DOMPurify
F-24	BAIXA	P3	termo-adesao-text.ts escapa placeholders dinâmicos
F-25	BAIXA	P3	Sanitização padronizada entre transparency e email-templates
F-26	INFORMATIVA	P3	Transparência usa rehype-sanitize para conteúdo Markdown

Pontos fracos centrais

- (1) F-19/F-20/F-21: Templates de email nao escapam HTML.** Cadeia de exploit: registro de conta com nome malicioso + clique em 'esqueci senha' = XSS no email. Combinado com F-22 (case 'simple' no email.service.ts) permite execucao de JS no webmail do destinatario. **Mitigacao URGENTE** antes de deploy em producao.
- (2) F-10: GET /tokens/:id eh IDOR direto** — qualquer usuario autenticado pode ler qualquer token pelo UUID. Vazamento em massa de dados de transacoes.
- (3) F-02/F-11: PUT /campaigns/:id/resources sem auth** — permite a qualquer visitante anonimo manipular alocoes de recursos de qualquer campanha.
- (4) F-14: JWT_SECRET com fallback hardcoded** — se operador esquecer de setar a env, sistema roda com secret publicado no codigo-fonte. Bypass total de auth.
- (5) F-16: Segredos REAIS (AWS, SMTP, EFI) em .env locais** — credenciais precisam ser rotacionadas e movidas para gerenciador de segredos (AWS Secrets Manager/Vault).

Pontos fortes verificados

- AuthGuard centralizado valida sessao em Redis com TTL 7 dias (auth.guard.ts:46-78). Sem cookie = 401. Sem sessao no Redis = 401. Usuario inativo = 401.
- Cookie session_id com flags httpOnly + sameSite=lax + secure em prod (cookies.service.ts:11-16). Nenhum token em localStorage.
- Sessao UUID v4 criptograficamente aleatorio (auth.service.ts:192). Colisao computacionalmente impossivel.
- TanStack Query: queryClient.clear() em login/register/logout (hooks/use-logout-mutation.ts:19, use-login-mutation.ts:47, use-register-mutation.ts:57). Sem vazamento de cache entre contas no mesmo navegador.
- Backend subscriptions.service.findOne/findAll/update/remove checam ownership via req.user.id (subscriptions.service.ts:382-411, 452-477). Sem IDOR no GET/DELETE de subscriptions.
- Backend payments.service.findOne (linha 326-394) e generatePix (linha 420-443) checam ownership. Sem IDOR no GET/POST payment.

- Backend investments.service (linha 199-426) checa ownership em confirm/cancel/getConfirmation.
- Backend notifications.controller usa req.user.id em todas as rotas (notifications.controller.ts:78-90, 109-115, 139-145, 156-162).

Pontos fortes verificados (continuacao)

A auditoria tambem confirma o que esta CORRETAMENTE protegido no sistema. Estes pontos nao precisam de acao imediata e devem ser mantidos em PRs futuras.

- Frontend (transparency/*) usa rehype-sanitize em todo o conteudo gerado pelo usuario. Sem XSS no forum de discussao.
- Routes admin (/admin/*) tem @UseGuards(AuthGuard, AdminGuard) em 13 controllers (admin.controller.ts, admin-other.controller.ts, admin-kyc.controller.ts, admin-seals.controller.ts, etc).
- Helmet CSP configurado (main.ts:39-79) com imgSrc permitindo S3/CloudFront (impede hotlinking de fontes externas nao autorizadas).
- Pino/Logger do NestJS usado em producao (sem console.log). Sem exposicao de secrets em logs.
- ThrottlerGuard global (main.ts implicito via APP_GUARD se configurado em algum modulo) — defesa contra brute force.
- Senhas armazenadas com bcrypt salt rounds=10 (auth.service.ts:402).
- BFFs (routes/api/*) propagam Set-Cookie e nao armazenam tokens. Cookies HTTP-only isolam tokens do JS do browser.
- EFI webhooks protegidos por WebhookSignatureGuard (pagamento/efi.controller.ts:59).
- BackupInterceptor cria snapshot antes de mutacoes destrutivas em users (backup/backup.interceptor.ts).
- AuditService registra mudancas de KYC, subscription cancel, etc. (audit/audit.service.ts).
- Prisma unique constraints em email e publicId (schema.sqlite.prisma:32-33).
- TLS 1.2+ forçado em producao (secure: process.env.NODE_ENV === 'production' no cookie).

Achados detalhados por categoria

Tabela abaixo: Severidade | Arquivo:linha | Descricao. Chip colorido a esquerda indica a severidade. Detalhes tecnicos completos estao em cada secao.

Categoria 1 — Isolamento por userId (queries sem filtro de dono)

CRITICA	F-02 — PUT /campaigns/:id/resources sem autenticao nem checagem de owner
Arquivo	backendnode/src/api/campaigns/campaigns.controller.ts:201-214
Evidencia	@Put('/:id/resources') async updateResources(@Param('id') id, @Body() dto) { return this.resources.replaceAll(+id, dto.resourceAllocations); }
Por que eh exploravel	Rota desprotegida (sem @UseGuards). Atacker anonimo pode chamar PUT /campaigns/123/resources com payload malicioso e SUBSTITUIR as alocoes de recursos de qualquer campanha — a unica protecao eh o @Body() ser validado por DTO.
Impacto	Manipulacao das alocoes de 100% de recursos de qualquer campanha. Quebra de regra de negocio B05/CVM 88/2022. Frontend deixa de exibir informacoes confiaveis.
Correcao sugerida	Adicionar @UseGuards(AuthGuard) e validar que req.user.id eh o founder da startup dona da campanha antes de replaceAll.

ALTA	F-03 — GET /startup/:id publico retorna email do founder (LGPLD)
Arquivo	backendnode/src/api/startup/service/startup-crud.service.ts:647-685
Evidencia	founder: { select: { id: true, nome: true, email: true } } (linha 658-660) – controller (startup.controller.ts:137-148) nao tem @UseGuards
Por que eh exploravel	Rota GET /startup/:id eh publica (sem guard). O payload inclui o email do founder. Qualquer visitante anonimo consegue listar founders da plataforma e seus emails.
Impacto	Violacao de LGPLD: exposicao de PII (email) sem consentimento ou necessidade. Possivel alvo de phishing/scraping.
Correcao sugerida	Remover `email: true` do select de founder. Se o email for necessario para CTAs, criar endpoint autenticado /api/startups/:id/contact que retorna mascarado e rate-limited.

MEDIA	F-04 — GET /campaigns e /campaigns/:id publicos sem filtro de status
Arquivo	backendnode/src/api/campaigns/service/campaigns-crud.service.ts:20-77 / 79-156
Evidencia	findAll aplica a allowlist PUBLIC_CAMPAIGN_STATUSES = ['OPEN', 'FUNDED', 'PAID_OUT']; findOne exige `id` e status nessa mesma allowlist. O endpoint administrativo protegido `/admin/compliance/campaigns` permite Compliance/Admin revisar campanhas DRAFT, PAUSED e CLOSED sem usar o catalogo publico.
Por que eh exploravel	Endpoint publico retorna campanhas nao publicadas quando nao restringe os status. Visitantes podem ver campanhas em rascunho com dados internos.

Impacto	Vazamento de informacoes de campanhas nao publicadas (incluindo dados bancarios da startup, dados de receita, projecoes).
Correcao sugerida	Manter o catalogo publico restrito a OPEN, FUNDED e PAID_OUT. Para revisao administrativa de DRAFT, PAUSED e CLOSED, usar GET /admin/compliance/campaigns e GET /admin/compliance/campaigns/:id com AuthGuard + AdminGuard; os BFFs de Compliance foram migrados para essas rotas.

MEDIA**F-05 — GET /campaigns/:id/resources sem filtro de status e projecao segura**

Arquivo	backendnode/src/api/campaigns/campaigns.controller.ts:188-219
Evidencia	A leitura publica valida status OPEN/FUNDED/PAID_OUT e seleciona apenas categoria, percentual e descricaoCustomizada. Campanhas DRAFT/PAUSED/CLOSED usam /campaigns/:id/resources/private com AuthGuard e verificacao de owner/papel administrativo.
Por que eh exploravel	A rota anterior consultava alocoes de qualquer campanha e retornava o row Prisma completo, incluindo IDs e timestamps internos. Isso permitia inferir estrategia de campanhas nao publicadas.
Impacto	Vazamento de estrategia de alocao e metadados internos de campanhas para visitantes e usuarios sem ownership.
Correcao sugerida	Manter a leitura publica apenas para campanhas OPEN/FUNDED/PAID_OUT com DTO minimo. Para campanhas nao publicadas, exigir AuthGuard e permitir somente o founder owner ou ADMIN, FINANCEIRO e COMPLIANCE.

BAIXA**F-06 — POST /payment aceita subscriptionId/investmentId/campaignId do body sem ownership**

Arquivo	backendnode/src/api/payment/payment.service.ts:111-190, 1795-1840
Evidencia	PaymentService.validatePaymentReferences() roda antes de payment.create() e de createCheckout(). SUBSCRIPTION consulta subscription.findFirst({ id, userId }); INVESTMENT consulta investment.findFirst({ id, userId }) e exige campaignId consistente quando informado; TOKEN_RESERVATION consulta campaign.findFirst({ id, startup: { founderId: userId } }); demais propositos rejeitam IDs relacionados. Referencias inexistentes ou de outra conta retornam REFERENCIA_PAGAMENTO_INVALIDA. A confirmacao de investimento tambem repassa payment.userId ao InvestmentsService.
Por que eh exploravel	User A podia enviar subscriptionId, investmentId ou campaignId de B no body. Sem validar o vinculo antes da criacao, o Payment era aceito com uma referencia pertencente a outra conta ou com combinacoes inconsistentes.
Impacto	Criacao de payments vinculados a recursos de outros usuarios, possibilidade de efeitos de dominio na conta errada e inconsistencias entre investimento e campanha.
Correcao sugerida	Validar ownership e combinacoes por purpose no servico, antes de inserir o Payment: assinatura pertence ao pagador; investimento pertence ao pagador; campaignId de INVESTMENT apenas coincide com o investimento; TOKEN_RESERVATION exige campanha da startup do founder; propósitos sem relacionamento rejeitam IDs extras. Cobertura unitária inclui cross-user, referencias invalidas, combinacoes inconsistentes e fluxos legitimis.

BAIXA

F-07 — GET /admin/config/fundraising e /admin/config/parameters acessíveis a qualquer user autenticado

Arquivo	backendnode/src/api/admin/admin.controller.ts:269-447; config.service.ts: public snapshot; frontend new-startup
Evidencia	AdminConfigController agora usa @UseGuards(AuthGuard, AdminGuard) no nível da classe, protegendo GET /admin/config/fundraising e GET /admin/config/parameters, além das escritas. O wizard de founder deixou de consumir o endpoint administrativo e usa /config/fundraising, protegido por AuthGuard, que retorna somente os campos operacionais necessários, sem histórico ou metadados administrativos.
Por que eh exploravel	Os GETs administrativos estavam acessíveis a qualquer usuário autenticado porque a classe tinha somente AuthGuard. Além do vazamento de configurações de negócio, o bloqueio direto poderia quebrar o wizard de founder, que dependia indevidamente do endpoint admin.
Impacto	Usuários comuns podiam consultar taxas, limites e histórico de parâmetros administrativos. O fallback silencioso do wizard também podia usar defaults frontend divergentes do backend.
Correcao sugerida	Adicionar AuthGuard + AdminGuard no nível da classe AdminConfigController e remover guards redundantes dos métodos. Criar leitura operacional autenticada em /config/fundraising com payload mínimo para founders, migrar BFF/loader/query do wizard e cobrir ADMIN/FINANCEIRO/COMPLIANCE, bloqueio de USER, contrato mínimo e propagação de 403.

Categoria 2 — Permissao definida no servidor (gates admin)

MEDIA

F-08 — AdminGuard usado sem AuthGuard — rotas admin inacessíveis OU 403 fixo

Arquivo	backendnode/src/api/payment/split/split.controller.ts:35-88
Evidencia	Auditoria global confirmou que a única ocorrência de @UseGuards(AdminGuard) isolado estava nos métodos create, update e deactivate do SplitController. O controller agora declara @UseGuards(AuthGuard, AdminGuard) na classe, na ordem explícita de autenticação e autorização, sem guards administrativos isolados nos métodos.
Por que eh exploravel	AdminGuard depende de request.user populado pelo AuthGuard. Quando a autorização era declarada separadamente no método, a dependência entre os guards ficava implícita e o padrão podia causar 403 incorreto para requisições sem autenticação ou ser replicado de forma insegura.
Impacto	As rotas administrativas de split passam a autenticar a sessão antes de validar ADMIN, FINANCEIRO ou COMPLIANCE, evitando bloqueio funcional de administradores válidos e mantendo USER sem autorização.
Correcao sugerida	Combinar @UseGuards(AuthGuard, AdminGuard) no nível da classe SplitController e remover decorators AdminGuard redundantes dos métodos. Regressão verifica a metadata na ordem exata; testes de AdminGuard cobrem ausência de usuário, USER e os três papéis administrativos.

BAIXA

F-09 — Frontend tem useUserRole()/hasRole() mas o backend eh a fonte de verdade — sem furo direto encontrado

Arquivo	AGENTS.md; frontend/AGENTS.md; frontend/app/hooks/use-user-role.ts:Frontend auth docs; use-user-role.ts:1
---------	---

Evidencia	A regra foi documentada no AGENTS.md raiz e em frontend/AGENTS.md: useUserRole()/hasRole() controlam somente affordances visuais. O hook agora declara explicitamente que nunca substitui autorização server-side; endpoints continuam protegidos por AuthGuard + AdminGuard/ComplianceGuard/FinanceRoleGuard. A regressão use-user-role.spec.ts cobre roles válidas, ausência de usuário e role desconhecida.
Por que eh exploravel	A prática correta já estava em uso, mas o limite de segurança entre UI e backend não estava explícito na documentação nem coberto por teste do hook. Sem essa regra, uma futura tela poderia tratar o gate visual como autorização ou omitir a revalidação server-side.
Impacto	Nenhum bypass imediato foi encontrado. A documentação e a regressão reduzem o risco de uma futura rota administrativa confiar em role derivada do navegador, DOM, estado local ou resposta mockada.
Correcao sugerida	Documentar que o gate de UI é cosmético e que toda permissão DEVE ser revalidada no backend com sessão HTTP-only, AuthGuard e guard de domínio apropriado. Manter o hook sem responsabilidade de autorização e cobrir sua derivação de role com teste unitário.

Categoria 3 — IDOR (acesso direto a objeto por ID sem ownership)

CRITICA**F-10 — GET /tokens/:id — IDOR permite ler qualquer token pelo UUID**

Arquivo	backendnode/src/api/tokens/tokens.controller.ts; tokens.service.ts:25-31; tokens.service.ts:139-190
Evidencia	A implementação atual extrai req.user.id no TokensController.detail() e chama getTokenById(id, req.user.id). O TokensService aplica findFirst({ where: { id: tokenId, userId } }), fazendo Token de outra conta resultar em 404 sem payload. A regressão tokens.service.spec.ts cobre owner e cross-user; tokens.controller.spec.ts confirma o encaminhamento do userId e metadata AuthGuard. AuthGuard também tem regressão para ausência de sessionId.
Por que eh exploravel	A evidência histórica descrevia uma consulta somente por id, que permitiria ao usuário A ler o Token do usuário B. O source atual já contém o filtro composto, mas a ausência de testes co-localizados deixava o contrato de ownership sem proteção contra regressão.
Impacto	Sem o filtro, haveria vazamento do hash, valores, certificado e histórico do Token. Com a implementação atual e a regressão adicionada, UUID de outra conta é indistinguível de token inexistente no detalhe privado.
Correcao sugerida	Preservar o filtro obrigatório por { id: tokenId, userId } no detalhe privado e o encaminhamento de req.user.id pelo controller. Não criar /admin/tokens/:id sem requisito de negócio; o certificado mantém regra explícita: owner ou ADMIN. Adicionar testes para owner, cross-user, AuthGuard, USER e ADMIN.

CRITICA**F-11 — PUT /campaigns/:id/resources — IDOR permite alterar recursos de qualquer campanha**

Arquivo	backendnode/src/api/campaigns/campaigns.controller.ts; campaigns/service/campaign-resource.service.ts:225-2
---------	---

Evidencia	A rota PUT /campaigns/:id/resources usa @UseGuards(AuthGuard), recebe req.user e encaminha a identidade ao service. CampaignResourceService.replaceAll busca startup.founderId antes da transação e permite somente o founder owner ou ADMIN; founder de outra startup, USER, FINANCEIRO e COMPLIANCE recebem NOT_OWNER sem delete/create. A regressão de controller confirma AuthGuard e encaminhamento; a regressão do service cobre owner, cross-owner, ADMIN, FINANCEIRO, COMPLIANCE, USER e campanha inexistente.
Por que eh exploravel	A evidência histórica descrevia uma rota sem autenticação que aceitava somente o campaignId enviado pelo cliente. Isso permitiria substituir as alocações de qualquer campanha. O source atual já valida a sessão no controller e ownership no service; o finding permaneceu aberto por falta de sincronização e cobertura explícita para todos os papéis.
Impacto	Sem a proteção, qualquer visitante ou founder de outra startup poderia manipular a estratégia de alocação de recursos. Com AuthGuard, ownership e teste de ausência de transação em falhas, a alteração cross-startup fica bloqueada para papéis não ADMIN.
Correcao sugerida	Preservar @UseGuards(AuthGuard) no PUT, encaminhar req.user e manter a checagem founderId ou role ADMIN antes de iniciar \$transaction. Cobrir anônimo via AuthGuard, owner legítimo, cross-owner, ADMIN, FINANCEIRO, COMPLIANCE, USER e campanha inexistente. Não alterar a regra de negócio atual que permite escrita cross-startup somente a ADMIN.

ALTA**F-12 — GET /startup/marketplace/private/:slug — sem checar se user eh founder/compliance/admin**

Arquivo	backendnode/src/api/startup/startup.controller.ts; startup.service.ts; startup-crud.service.ts:109-144; 58-66; 520-6
Evidencia	O endpoint owner/admin usa @UseGuards(AuthGuard), recebe req.user e chama findPrivate(slug, req.user). O StartupCrudService aplica founderId = request.user.id para founders e permite somente ADMIN, FINANCEIRO e COMPLIANCE sem esse filtro. A descoberta de investidores foi separada em /marketplace/authenticated/:slug, também protegida por AuthGuard, com payload mínimo sem id interno ou PII. Testes cobrem owner, cross-user, papéis administrativos, USER/INVESTOR, inexistente e metadata dos guards.
Por que eh exploravel	A evidência histórica mostrava que qualquer sessão válida podia consultar qualquer slug pelo endpoint chamado private. Isso permitia acesso cross-user ao detalhe sem checagem de ownership. A correção preserva o fluxo legítimo de descoberta antes do investimento em endpoint autenticado separado e restringe a superfície private ao owner/admin no servidor.
Impacto	Sem a proteção, um founder poderia consultar o detalhe privado de startup de outro founder. Com a autorização por founderId, o cross-user retorna 404 sem payload; papéis administrativos continuam podendo revisar startups, e investidores autenticados recebem somente o contrato mínimo necessário para análise e checkout.
Correcao sugerida	Passar request.user ao service, filtrar Startup.founderId para o endpoint private e permitir apenas ADMIN, FINANCEIRO e COMPLIANCE como bypass administrativo. Usar endpoint autenticado separado para investidores, sem id interno, dados bancários, email, CNPJ ou documento do titular; manter preview do founder dependente do endpoint owner/admin.

MEDIA

F-13 — GET /campaigns/:id — projeção pública expunha dados completos da campanha

Arquivo	backendnode/src/api/campaigns/service/campaigns-crud.service.ts; campaigns.controller.ts:88-157; 56-61
Evidencia	O detalhe público mantém a allowlist server-side de status OPEN, FUNDED e PAID_OUT; DRAFT, PAUSED e CLOSED resultam em 404. A consulta agora usa select explícito e o mapper retorna somente id/título/status/valores públicos, deadline, startup com nome/slug/área/estágio/logo e métricas derivadas. Não são retornados investimentos individuais, tokens, hashes, CNPJ, razão social, descrição ou campos internos. A revisão administrativa continua em /admin/compliance/campaigns/:id com AuthGuard + AdminGuard. Testes cobrem projeção mínima, status não públicos e separação administrativa.
Por que eh exploravel	A evidência histórica tratava o endpoint como capaz de retornar qualquer campanha e o registro completo. O filtro de status já existia no source atual, mas a resposta ainda fazia over-fetching com investimentos, tokens e campos sensíveis da startup. A correção preserva o detalhe público para campanhas publicadas e transforma o retorno em uma projeção explícita.
Impacto	Sem a projeção, visitantes poderiam coletar linhas de investimento, tokens, hashes e dados cadastrais/operacionais de campanhas publicadas; campanhas não publicadas também poderiam ser alvo de enumeração se a allowlist regredisse. Com a consulta filtrada e o payload mínimo, o catálogo público não expõe ativos, transações individuais ou dados administrativos.
Correcao sugerida	Manter a allowlist OPEN/FUNDED/PAID_OUT no detalhe público e substituir include/spread do registro por select e mapper explícitos. Remover investimentos, tokens/hashes, CNPJ, razão social, descrição e campos internos do contrato público; manter o endpoint administrativo separado para revisão completa de qualquer status.

Categoria 4 — Chaves expostas / hardcoded

CRITICA

F-14 — JWT_SECRET sem fallback e validado no bootstrap

Arquivo	backendnode/src/auth/auth.module.ts; backendnode/src/common/config/env.schema.ts:24-30; 25-33
Evidencia	AuthModule injeta ConfigService e lê JWT_SECRET sem valor alternativo; envSchema exige segredo presente com no mínimo 32 caracteres e rejeita placeholders conhecidos. app.module.ts executa envSchema.parse(config) durante o bootstrap.
Por que eh exploravel	A implementação anterior permitia inicializar a aplicação com uma chave JWT conhecida quando a variável não era configurada. A configuração atual falha de forma explícita antes de servir requisições.
Impacto	O fallback previsível foi removido; a falsificação de tokens por ausência de segredo deixa de ser possível no caminho de inicialização validado.
Correcao sugerida	Remover o fallback, centralizar a validação em envSchema e cobrir ausência, tamanho mínimo, vazio e placeholders com testes unitários.

ALTA

F-15 — WEBHOOK_HASH_SECRET obrigatório e sem fallback hardcoded

Arquivo	backendnode/src/api/webhook/webhook-log.service.ts; backendnode/src/common/config/env.schema.ts:73-77; 3
---------	--

Evidencia	WebhookLogService usa ConfigService.getOrThrow('WEBHOOK_HASH_SECRET'); envSchema exige segredo com no mínimo 32 caracteres e rejeita placeholders conhecidos. O teste verifica HMAC-SHA256 com segredo injetado e falha quando ausente.
Por que eh exploravel	A implementação anterior usava uma chave conhecida quando WEBHOOK_HASH_SECRET não estava configurado, permitindo correlação de PII anonimizada nos logs. O serviço agora falha sem configuração válida.
Impacto	A anonimização HMAC de PII em logs de webhook não depende mais de um segredo publicado ou implícito.
Correcao sugerida	Remover o fallback, separar o segredo de JWT/EFI, exigir configuração no schema e cobrir o comportamento com teste unitário.

ALTA**F-16 — Credenciais operacionais exigem rotação e secret manager**

Arquivo	backendnode/.env, backendnode/.env.prod, infra/prod/ec2-api/user-data.sh:global
Evidencia	Os arquivos de ambiente versionados localmente foram sanitizados e o provisionamento deixou de conter valores de AWS/SMTP/EFI/JWT; user-data lê /etc/iselftoken/ec2-api.env com permissões root-only. Não há evidência operacional de revogação/rotação dos valores que existiram fora do código nem de migração para um secret manager.
Por que eh exploravel	Remover valores do código e exigir injeção externa reduz exposição futura, mas não invalida credenciais que possam ter sido copiadas ou comprometidas anteriormente.
Impacto	AWS, SMTP e EFI permanecem dependentes de rotação coordenada com os provedores e de confirmação do armazenamento seguro em produção.
Correcao sugerida	Pendente: revogar e gerar novas credenciais nos provedores, atualizar o secret manager/arquivo root-only de produção, validar o deploy e remover cópias operacionais antigas após confirmação.

MEDIA**F-17 — EFI_WEBHOOK_HMAC_SECRET depende de rotação operacional**

Arquivo	backendnode/.env, backendnode/.env.prod, infra/prod/ec2-api/user-data.sh:global
Evidencia	Os exemplos e o provisionamento não carregam um valor de EFI_WEBHOOK_HMAC_SECRET; o user-data exige a variável na fonte externa de secrets. A rotação no painel/provedor EFI e a confirmação de revogação do valor anterior ainda não foram executadas.
Por que eh exploravel	A configuração deixou de publicar um valor de desenvolvimento no fluxo de provisionamento, mas a correção operacional não pode ser comprovada apenas pelo repositório.
Impacto	Até a rotação confirmada, um valor antigo eventualmente compartilhado pode permitir falsificação de callbacks EFI.
Correcao sugerida	Pendente: gerar HMAC forte, atualizar a configuração da EFI e da produção, revogar o valor anterior e executar teste de assinatura controlado.

BAIXA**F-18 — VITE_EFI_PAYEE_CODE público por design (informativo)**

Arquivo	frontend/.env, frontend/.env.prod:global
----------------	--

Evidencia	VITE_EFI_PAYEE_CODE é embutido no bundle por ser uma variável pública do frontend; o payee_code não é credencial segundo o contrato da EFI. Nenhuma chave privada ou HMAC é derivada desse valor.
Por que eh exploravel	Variáveis VITE_* são visíveis no navegador. Este achado não representa segredo exposto, mas a natureza pública deve permanecer documentada para evitar uso indevido.
Impacto	Nenhum impacto de confidencialidade identificado; o código público é esperado para iniciar pagamentos destinados ao recebedor configurado.
Correcao sugerida	Manter a documentação explícita de que payee_code é público e não reutilizá-lo como segredo de autenticação.

Categoria 5 — Inputs sem tratamento (XSS / template injection)

CRITICA

F-19 — Templates de email escapam dados e validam links dinâmicos

Arquivo	backendnode/src/email/templates/verification-code.template.ts; backendnode/src/api/email-templates/email-render
Evidencia	O template de código aplica escapeHtml a nome, ação, código e validade, usa escapeSafeUrl para redirectPath e mantém href entre aspas. O renderer de templates escapa placeholders HTML, valida href/src dinâmicos e mantém texto/assunto em contexto textual.
Por que eh exploravel	O vetor histórico combinava interpolação de dados do usuário sem escaping e redirectPath sem validação. A cadeia foi fechada nos templates hardcoded e no renderer de templates persistidos.
Impacto	Scripts, handlers e javascript: URLs fornecidos nos dados não são renderizados como HTML executável nem aceitos como links dinâmicos.
Correcao sugerida	Aplicar escaping contextual nos templates, validar URLs antes do escaping, usar atributos href citados e cobrir payloads XSS/URLs inseguras nos testes de renderer e templates.

ALTA

F-20 — Templates welcome e forgot-password escapam dados do usuário

Arquivo	backendnode/src/email/templates/welcome.template.ts, forgot-password.template.ts:9-52 / 10-62
Evidencia	welcomeTemplate aplica escapeHtml a nome e email; forgotPasswordTemplate aplica escapeHtml a nome, código e validade antes de montar o HTML.
Por que eh exploravel	Os templates de boas-vindas e recuperação eram parte do mesmo vetor de interpolação HTML sem escaping identificado na auditoria.
Impacto	Dados maliciosos nesses fluxos são tratados como texto, sem execução de markup ou handlers no cliente de email.
Correcao sugerida	Aplicar escaping HTML contextual em todas as interpolações dinâmicas dos templates e manter regressões para payloads de markup.

ALTA

F-21 — EmailRenderService usa escaping contextual no HTML

Arquivo	backendnode/src/api/email-templates/email-render.service.ts:13-26; 100-124
---------	--

Evidencia	interpolate recebe escapeValues; render chama interpolate(..., true) para HTML e false para texto/assunto. Valores ausentes permanecem como placeholders e URLs href/src dinâmicas são validadas antes da interpolação.
Por que eh exploravel	A função de escaping deixou de ser código morto: o renderer agora aplica o tratamento adequado ao contexto de saída.
Impacto	Templates persistidos não inserem dados de usuário diretamente no HTML sem escaping; texto e assunto não recebem escaping HTML indevido.
Correcao sugerida	Refatorar a interpolação para distinguir contexto HTML de texto, validar URLs dinâmicas antes do escaping e cobrir campos desconhecidos/obrigatórios.

ALTA**F-22 — EmailService escapa texto simples antes de convertê-lo em HTML**

Arquivo	backendnode/src/email/email.service.ts:279-284
Evidencia	No caso simple, o serviço usa <code><p>\${escapeHtml(text).replace(/\n/g, '
')}</code> e mantém o conteúdo original apenas no campo text do email.
Por que eh exploravel	A interpolação direta de text em um parágrafo HTML permitia que callers transformassem markup em conteúdo executável.
Impacto	Payloads HTML enviados como texto simples são serializados como texto no HTML do email.
Correcao sugerida	Aplicar escapeHtml ao texto antes de inserir a marcação de quebra de linha e cobrir script/markup nos testes de EmailService.

MEDIA**F-23 — Previews HTML de templates usam DOMPurify**

Arquivo	frontend/app/lib/sanitize-html.ts; frontend/app/components/admin/email-templates/email-template-preview-modal
Evidencia	sanitizeHtmlPreview usa DOMPurify.sanitize com configuração explícita de tags/atributos permitidos. O helper é aplicado tanto ao dangerouslySetInnerHTML do modal quanto ao preview local do editor.
Por que eh exploravel	Os dois sinks frontend que renderizam HTML bruto foram tratados; não há confiança direta no HTML vindo do renderer de templates.
Impacto	Markup, scripts, event handlers e protocolos perigosos são removidos antes da renderização no painel administrativo.
Correcao sugerida	Adicionar DOMPurify fixado, centralizar a sanitização em helper reutilizável e cobrir os dois sinks com regressões de XSS e markup seguro.

BAIXA**F-24 — termo-adesao-text.ts escapa placeholders dinâmicos**

Arquivo	frontend/app/lib/termo-adesao-text.ts:1-45
Evidencia	fill aplica escapeHtml aos campos founderName, founderCpf, startupName, startupCnpj e companyEmail antes de substituir placeholders; o texto fixo do termo permanece intacto.
Por que eh exploravel	A função aceitava dados futuros do usuário e os inseria sem proteção contextual, criando um vetor adormecido em documento legal.
Impacto	Valores dinâmicos com markup são tratados como texto no termo; a substituição futura não reabre XSS por interpolação direta.

Correcao sugerida	Escapar cada valor dinâmico no contexto HTML antes da substituição e cobrir scripts, atributos e placeholders nos testes unitários.
-------------------	---

BAIXA

F-25 — Sanitização padronizada entre transparency e email-templates

Arquivo	frontend/app/components/transparency/*; frontend/app/lib/sanitize-html.ts; email-template-preview-modal.tsx; email-template-preview-modal.tsx; email-template-preview-modal.tsx
Evidencia	Transparency mantém rehype-sanitize para Markdown; os sinks HTML bruto de email-templates usam sanitizeHtmlPreview baseado em DOMPurify. Cada pipeline usa o sanitizer adequado ao formato de entrada.
Por que eh exploravel	A inconsistência histórica era a ausência de sanitização nos previews HTML, não uma necessidade de substituir o pipeline Markdown existente.
Impacto	Conteúdo Markdown e HTML bruto não são renderizados sem a camada de sanitização correspondente.
Correcao sugerida	Preservar rehype-sanitize no Markdown e aplicar DOMPurify centralizado em todos os previews HTML bruto.

INFORMATIVA

F-26 — Transparência usa rehype-sanitize para conteúdo Markdown

Arquivo	frontend/app/components/transparency/*:varia
Evidencia	discussion-replies, post-detail e transparency-featured-report importam rehype-sanitize e o utilizam no pipeline ReactMarkdown antes de renderizar conteúdo gerado pelo usuário.
Por que eh exploravel	Este achado é um controle positivo da auditoria e permanece válido após a adição do sanitizer específico para previews HTML bruto.
Impacto	Conteúdo Markdown de posts e replies permanece mitigado contra markup e scripts não permitidos.
Correcao sugerida	Nenhuma alteração necessária; manter rehype-sanitize e sua cobertura de uso nos componentes de transparência.

Recomendacoes priorizadas

Lista de acoes ordenadas por impacto. P1 = corrigir antes do proximo deploy. P2 = corrigir na proxima sprint. P3 = melhorar continuamente.

P1 — corrigir antes do proximo deploy (3 acoes)

- F-19/20/21 (XSS email): Aplicar `escapeHtml()` em TODAS interpolacoes nos templates. Trocar template engine para um com auto-escape (handlebars). Validar `redirectPath` com `@IsUrl`.
- F-14 (JWT_SECRET fallback): Remover o fallback em `auth.module.ts`. Lançar erro fatal no bootstrap se `process.env.JWT_SECRET` for undefined.
- F-16 (Secrets reais em .env): Rotacionar AWS keys, SMTP password e EFI secrets AGORA. Gerar novo JWT_SECRET com `openssl rand -base64 48`.

P2 — corrigir na proxima sprint (4 acoes)

- F-15 (WEBHOOK_HASH_SECRET fallback): Remover o fallback em `webhook-log.service.ts`.
- F-17 (HMAC EFI fraco): Gerar HMAC secret forte para EFI.
- F-22 (email.service case simple): Aplicar `escapeHtml()` antes de quebrar linha.
- F-23 (preview email sem sanitize): Adicionar `DOMPurify` antes do `dangerouslySetInnerHTML`.

P3 — melhorias continuas (6 acoes)

- F-24 (termo-adesao bode expirado): Aplicar `escapeHtml()` em `fill()` preventivamente.
- F-25 (consistencia sanitize): Padronizar uso de `DOMPurify/rehype-sanitize` em todo `dangerouslySetInnerHTML`.
- Implementar validacao de startup que rejeite defaults (`process.env.JWT_SECRET !== 'default-secret-change-in-production'` em prod).
- Migrar de .env para AWS Secrets Manager ou HashiCorp Vault.
- Adicionar testes E2E que cubram: 'User A nao consegue alterar resource allocation de B', etc.
- Implementar CSP nonce em `main.ts` para scripts inline.

ISSUES PARA O GITHUB

Abaixo estao issues prontas para copiar/colar no GitHub. Cada uma tem titulo, labels, descricao, evidencia, impacto, sugestao de correcao e criterios de aceite.

ISSUE 1 (F-02)

--- ISSUE 1 ---

Titulo: [Seguranca] [CRITICA] PUT /campaigns/:id/resources sem autenticacao nem checagem de owner

Labels: security, critica

Descricao

Rota desprotegida (sem @UseGuards). Atacker anonimo pode chamar PUT /campaigns/123/resources com payload malicioso e SUBSTITUIR

Evidencia

Arquivo: `backendnode/src/api/campaigns/campaigns.controller.ts:201-214`

```
@Put('/:id/resources') async updateResources(@Param('id') id, @Body() dto) { return this.resources.replaceAll(+id, dto.resources
```

Impacto

Manipulacao das alocoes de 100% de recursos de qualquer campanha. Quebra de regra de negocio B05/CVM 88/2022. Frontend deixa

Sugestao de correcao

Adicionar @UseGuards(AuthGuard) e validar que req.user.id eh o founder da startup dona da campanha antes de replaceAll.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 1 ---

ISSUE 2 (F-03)

--- ISSUE 2 ---

Titulo: [Seguranca] [ALTA] GET /startup/:id publico retorna email do founder (LGPD)

Labels: security, alta

Descricao

Rota GET /startup/:id eh publica (sem guard). O payload inclui o email do founder. Qualquer visitante anonimo consegue listar fo

Evidencia

Arquivo: `backendnode/src/api/startup/service/startup-crud.service.ts:647-685`

```
founder: { select: { id: true, nome: true, email: true } } (linha 658-660) - controller (startup.controller.ts:137-148) nao t
```

Impacto

Violacao de LGPD: exposicao de PII (email) sem consentimento ou necessidade. Possivel alvo de phishing/scraping.

Sugestao de correcao

Remover `email: true` do select de founder. Se o email for necessario para CTAs, criar endpoint autenticado /api/startups/:id/co

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 2 ---

ISSUE 3 (F-04)

--- ISSUE 3 ---

Titulo: [Seguranca] [MEDIA] GET /campaigns e /campaigns/:id publicos sem filtro de status

Labels: security, media

```
## Descricao
Endpoint publico retorna campanhas nao publicadas quando nao restringe os status. Visitantes podem ver campanhas em rascunho cor

## Evidencia
Arquivo: `backendnode/src/api/campaigns/service/campaigns-crud.service.ts:20-77 / 79-156`

    findAll aplica a allowlist PUBLIC_CAMPAIGN_STATUSES = ['OPEN', 'FUNDED', 'PAID_OUT']; findOne exige `id` e status nessa mesma

## Impacto
Vazamento de informacoes de campanhas nao publicadas (incluindo dados bancarios da startup, dados de receita, projecoes).

## Sugestao de correcao
Manter o catalogo publico restrito a OPEN, FUNDED e PAID_OUT. Para revisao administrativa de DRAFT, PAUSED e CLOSED, usar GET /a

## Critérios de aceite
- [ ] Teste de regressao que prova o bug existe antes do fix (RED)
- [ ] Fix aplicado
- [ ] Teste de regressao passa (GREEN)
- [ ] Auditoria manual confirma que o vetor esta fechado
- [ ] PR revisado por outro dev
- [ ] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 3 ---
```

ISSUE 4 (F-05)

```
--- ISSUE 4 ---

Titulo: [Seguranca] [MEDIA] GET /campaigns/:id/resources sem filtro de status e projecao segura

Labels: security, media

## Descricao
A rota anterior consultava alocacoes de qualquer campanha e retornava o row Prisma completo, incluindo IDs e timestamps internos

## Evidencia
Arquivo: `backendnode/src/api/campaigns/campaigns.controller.ts:188-219`

    A leitura publica valida status OPEN/FUNDED/PAID_OUT e seleciona apenas categoria, percentual e descricaoCustomizada. Campanha

## Impacto
Vazamento de estrategia de alocao e metadados internos de campanhas para visitantes e usuarios sem ownership.

## Sugestao de correcao
Manter a leitura publica apenas para campanhas OPEN/FUNDED/PAID_OUT com DTO minimo. Para campanhas nao publicadas, exigir AuthG

## Critérios de aceite
- [ ] Teste de regressao que prova o bug existe antes do fix (RED)
- [ ] Fix aplicado
- [ ] Teste de regressao passa (GREEN)
- [ ] Auditoria manual confirma que o vetor esta fechado
- [ ] PR revisado por outro dev
- [ ] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 4 ---
```

ISSUE 5 (F-06)

```
--- ISSUE 5 ---

Titulo: [Seguranca] [BAIXA] POST /payment aceita subscriptionId/investmentId/campaignId do body sem ownership

Labels: security, baixa

## Descricao
User A podia enviar subscriptionId, investmentId ou campaignId de B no body. Sem validar o vinculo antes da criacao, o Payment c

## Evidencia
Arquivo: `backendnode/src/api/payment/payment.service.ts:111-190, 1795-1840`

    PaymentService.validatePaymentReferences() roda antes de payment.create() e de createCheckout(). SUBSCRIPTION consulta subscr

## Impacto
Criacao de payments vinculados a recursos de outros usuarios, possibilidade de efeitos de dominio na conta errada e inconsistenc

## Sugestao de correcao
Validar ownership e combinacoes por purpose no servico, antes de inserir o Payment: assinatura pertence ao pagador; investimento

## Critérios de aceite
```

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 5 ---

ISSUE 6 (F-07)

--- ISSUE 6 ---

Titulo: [Seguranca] [BAIXA] GET /admin/config/fundraising e /admin/config/parameters acessíveis a qualquer user autenticado

Labels: security, baixa

Descricao

Os GETs administrativos estavam acessíveis a qualquer usuário autenticado porque a classe tinha somente AuthGuard. Além do vazamento de dados, o sistema não estava protegido contra ataques de força bruta.

Evidencia

Arquivo: `backendnode/src/api/admin/admin.controller.ts:269-447; config.service.ts: public snapshot; frontend new-startup-loader`

AdminConfigController agora usa @UseGuards(AuthGuard, AdminGuard) no nível da classe, protegendo GET /admin/config/fundraising e /admin/config/parameters.

Impacto

Usuários comuns podiam consultar taxas, limites e histórico de parâmetros administrativos. O fallback silencioso do wizard tambem era acessível.

Sugestao de correcao

Adicionar AuthGuard + AdminGuard no nível da classe AdminConfigController e remover guards redundantes dos métodos. Criar leitura de logs para identificar tentativas de acesso não autorizado.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 6 ---

ISSUE 7 (F-08)

--- ISSUE 7 ---

Titulo: [Seguranca] [MEDIA] AdminGuard usado sem AuthGuard – rotas admin inacessíveis OU 403 fixo

Labels: security, media

Descricao

AdminGuard depende de request.user populado pelo AuthGuard. Quando a autorização era declarada separadamente no método, a dependência não era atendida, resultando em rotas inacessíveis ou erro 403.

Evidencia

Arquivo: `backendnode/src/api/payment/split/split.controller.ts:35-88`

Auditoria global confirmou que a única ocorrência de @UseGuards(AdminGuard) isolado estava nos métodos create, update e deactivate.

Impacto

As rotas administrativas de split passam a autenticar a sessão antes de validar ADMIN, FINANCEIRO ou COMPLIANCE, evitando bloqueios de acesso.

Sugestao de correcao

Combinar @UseGuards(AuthGuard, AdminGuard) no nível da classe SplitController e remover decorators AdminGuard redundantes dos métodos.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 7 ---

ISSUE 8 (F-09)

--- ISSUE 8 ---

Titulo: [Seguranca] [BAIXA] Frontend tem useUserRole()/hasRole() mas o backend eh a fonte de verdade – sem furo direto encontrado

Labels: security, baixa

Descricao

A prática correta já estava em uso, mas o limite de segurança entre UI e backend não estava explícito na documentação nem coberto

Evidencia

Arquivo: `AGENTS.md; frontend/AGENTS.md; frontend/app/hooks/use-user-role.ts:Frontend auth docs; use-user-role.ts:1-35`

A regra foi documentada no AGENTS.md raiz e em frontend/AGENTS.md: useUserRole()/hasRole() controlam somente affordances visuais

Impacto

Nenhum bypass imediato foi encontrado. A documentação e a regressão reduzem o risco de uma futura rota administrativa confiar em

Sugestao de correcao

Documentar que o gate de UI é cosmético e que toda permissão DEVE ser revalidada no backend com sessão HTTP-only, AuthGuard e g

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 8 ---

ISSUE 9 (F-10)

--- ISSUE 9 ---

Titulo: [Seguranca] [CRITICA] GET /tokens/:id – IDOR permite ler qualquer token pelo UUID

Labels: security, critica

Descricao

A evidência histórica descrevia uma consulta somente por id, que permitiria ao usuário A ler o Token do usuário B. O source atual

Evidencia

Arquivo: `backendnode/src/api/tokens/tokens.controller.ts; tokens.service.ts:25-31; tokens.service.ts:139-190`

A implementação atual extrai req.user.id no TokensController.detail() e chama getTokenById(id, req.user.id). O TokensService a

Impacto

Sem o filtro, haveria vazamento do hash, valores, certificado e histórico do Token. Com a implementação atual e a regressão adi

Sugestao de correcao

Preservar o filtro obrigatório por { id: tokenId, userId } no detalhe privado e o encaminhamento de req.user.id pelo controller

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 9 ---

ISSUE 10 (F-11)

--- ISSUE 10 ---

Titulo: [Seguranca] [CRITICA] PUT /campaigns/:id/resources – IDOR permite alterar recursos de qualquer campanha

Labels: security, critica

Descricao

A evidência histórica descrevia uma rota sem autenticação que aceitava somente o campaignId enviado pelo cliente. Isso permitir

Evidencia

Arquivo: `backendnode/src/api/campaigns/campaigns.controller.ts; campaigns/service/campaign-resource.service.ts:225-240; campai

A rota PUT /campaigns/:id/resources usa @UseGuards(AuthGuard), recebe req.user e encaminha a identidade ao service. CampaignR

Impacto

Sem a proteção, qualquer visitante ou founder de outra startup poderia manipular a estratégia de alocação de recursos. Com Auth

Sugestao de correcao

Preservar @UseGuards(AuthGuard) no PUT, encaminhar req.user e manter a checagem founderId ou role ADMIN antes de iniciar \$trans

```
## Critérios de aceite
- [ ] Teste de regressao que prova o bug existe antes do fix (RED)
- [ ] Fix aplicado
- [ ] Teste de regressao passa (GREEN)
- [ ] Auditoria manual
confirma que o vetor esta fechado
- [ ] PR revisado por outro dev
- [ ] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 10 ---
```

ISSUE 11 (F-12)

--- ISSUE 11 ---

Titulo: [Seguranca] [ALTA] GET /startup/marketplace/private/:slug – sem checar se user eh founder/compliance/admin

Labels: security, alta

Descricao

A evidência histórica mostrava que qualquer sessão válida podia consultar qualquer slug pelo endpoint chamado private. Isso per

Evidencia

Arquivo: `backendnode/src/api/startup/startup.controller.ts; startup.service.ts; startup-crud.service.ts:109-144; 58-66; 520-66`

O endpoint owner/admin usa @UseGuards(AuthGuard), recebe req.user e chama findPrivate(slug, req.user). O StartupCrudService ap

Impacto

Sem a proteção, um founder poderia consultar o detalhe privado de startup de outro founder. Com a autorização por founderId, o c

Sugestao de correcao

Passar request.user ao service, filtrar Startup(founderId para o endpoint private e permitir apenas ADMIN, FINANCEIRO e COMPLIA

Critérios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 11 ---

ISSUE 12 (F-13)

--- ISSUE 12 ---

Titulo: [Seguranca] [MEDIA] GET /campaigns/:id – projeção pública expunha dados completos da campanha

Labels: security, media

Descricao

A evidência histórica tratava o endpoint como capaz de retornar qualquer campanha e o registro completo. O filtro de status já e

Evidencia

Arquivo: `backendnode/src/api/campaigns/service/campaigns-crud.service.ts; campaigns.controller.ts:88-157; 56-61`

O detalhe público mantém a allowlist server-side de status OPEN, FUNDED e PAID_OUT; DRAFT, PAUSED e CLOSED resultam em 404. A

Impacto

Sem a projeção, visitantes poderiam coletar linhas de investimento, tokens, hashes e dados cadastrais/operacionais de campanhas

Sugestao de correcao

Manter a allowlist OPEN/FUNDED/PAID_OUT no detalhe público e substituir include/spread do registro por select e mapper explícit

Critérios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 12 ---

ISSUE 13 (F-14)

--- ISSUE 13 ---

Titulo: [Seguranca] [CRITICA] JWT_SECRET sem fallback e validado no bootstrap

Labels: security, critica

Descricao

A implementação anterior permitia inicializar a aplicação com uma chave JWT conhecida quando a variável não era configurada. A

Evidencia

Arquivo: `backendnode/src/auth/auth.module.ts; backendnode/src/common/config/env.schema.ts:24-30; 25-33`

AuthModule injeta ConfigService e lê JWT_SECRET sem valor alternativo; envSchema exige segredo presente com no mínimo 32 caracte

Impacto

O fallback previsível foi removido; a falsificação de tokens por ausência de segredo deixa de ser possível no caminho de inicia

Sugestao de correcao

Remover o fallback, centralizar a validação em envSchema e cobrir ausência, tamanho mínimo, vazio e placeholders com testes unit

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 13 ---

ISSUE 14 (F-15)

--- ISSUE 14 ---

Titulo: [Seguranca] [ALTA] WEBHOOK_HASH_SECRET obrigatório e sem fallback hardcoded

Labels: security, alta

Descricao

A implementação anterior usava uma chave conhecida quando WEBHOOK_HASH_SECRET não estava configurado, permitindo correlação de f

Evidencia

Arquivo: `backendnode/src/api/webhook/webhook-log.service.ts; backendnode/src/common/config/env.schema.ts:73-77; 35-43`

WebhookLogService usa ConfigService.getOrThrow('WEBHOOK_HASH_SECRET'); envSchema exige segredo com no mínimo 32 caracteres e

Impacto

A anonimização HMAC de PII em logs de webhook não depende mais de um segredo publicado ou implícito.

Sugestao de correcao

Remover o fallback, separar o segredo de JWT/EFI, exigir configuração no schema e cobrir o comportamento com teste unitário.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 14 ---

ISSUE 15 (F-16)

--- ISSUE 15 ---

Titulo: [Seguranca] [ALTA] Credenciais operacionais exigem rotação e secret manager

Labels: security, alta

Descricao

Remover valores do código e exigir injeção externa reduz exposição futura, mas não invalida credenciais que possam ter sido cop

Evidencia

Arquivo: `backendnode/.env, backendnode/.env.prod, infra/prod/ec2-api/user-data.sh:global`

Os arquivos de ambiente versionados localmente foram sanitizados e o provisionamento deixou de conter valores de AWS/SMTP/EFI.

Impacto
AWS, SMTP e EFI permanecem dependentes de rotação coordenada com os provedores e de confirmação do armazenamento seguro em produção.

Sugestao de correcao
Pendente: revogar e gerar novas credenciais nos provedores, atualizar o secret manager/arquivo root-only de produção, validar o

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 15 ---

ISSUE 16 (F-17)

--- ISSUE 16 ---

Titulo: [Seguranca] [MEDIA] EFI_WEBHOOK_HMAC_SECRET depende de rotação operacional

Labels: security, media

Descricao

A configuração deixou de publicar um valor de desenvolvimento no fluxo de provisionamento, mas a correção operacional não pode s

Evidencia

Arquivo: `backendnode/.env, backendnode/.env.prod, infra/prod/ec2-api/user-data.sh:global`

Os exemplos e o provisionamento não carregam um valor de EFI_WEBHOOK_HMAC_SECRET; o user-data exige a variável na fonte externa

Impacto

Até a rotação confirmada, um valor antigo eventualmente compartilhado pode permitir falsificação de callbacks EFI.

Sugestao de correcao

Pendente: gerar HMAC forte, atualizar a configuração da EFI e da produção, revogar o valor anterior e executar teste de assinatura

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 16 ---

ISSUE 17 (F-18)

--- ISSUE 17 ---

Titulo: [Seguranca] [BAIXA] VITE_EFI_PAYEE_CODE público por design (informativo)

Labels: security, baixa

Descricao

Variáveis VITE_* são visíveis no navegador. Este achado não representa segredo exposto, mas a natureza pública deve permanecer c

Evidencia

Arquivo: `frontend/.env, frontend/.env.prod:global`

VITE_EFI_PAYEE_CODE é embutido no bundle por ser uma variável pública do frontend; o payee_code não é credencial segundo o co

Impacto

Nenhum impacto de confidencialidade identificado; o código público é esperado para iniciar pagamentos destinados ao recebedor co

Sugestao de correcao

Manter a documentação explícita de que payee_code é público e não reutilizá-lo como segredo de autenticação.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev

- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 17 ---

ISSUE 18 (F-19)

--- ISSUE 18 ---

Titulo: [Seguranca] [CRITICA] Templates de email escapam dados e validam links dinâmicos

Labels: security, critica

Descricao

O vetor histórico combinava interpolação de dados do usuário sem escaping e redirectPath sem validação. A cadeia foi fechada nos

Evidencia

Arquivo: `backendnode/src/email/templates/verification-code.template.ts; backendnode/src/api/email-templates/email-render.service.ts`

O template de código aplica escapeHtml a nome, ação, código e validade, usa escapeSafeUrl para redirectPath e mantém href ent

Impacto

Scripts, handlers e javascript: URLs fornecidos nos dados não são renderizados como HTML executável nem aceitos como links dinâm

Sugestao de correcao

Aplicar escaping contextual nos templates, validar URLs antes do escaping, usar atributos href citados e cobrir payloads XSS/URL

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 18 ---

ISSUE 19 (F-20)

--- ISSUE 19 ---

Titulo: [Seguranca] [ALTA] Templates welcome e forgot-password escapam dados do usuário

Labels: security, alta

Descricao

Os templates de boas-vindas e recuperação eram parte do mesmo vetor de interpolação HTML sem escaping identificado na auditoria

Evidencia

Arquivo: `backendnode/src/email/templates/welcome.template.ts, forgot-password.template.ts:9-52 / 10-62`

welcomeTemplate aplica escapeHtml a nome e email; forgotPasswordTemplate aplica escapeHtml a nome, código e validade antes de

Impacto

Dados maliciosos nesses fluxos são tratados como texto, sem execução de markup ou handlers no cliente de email.

Sugestao de correcao

Aplicar escaping HTML contextual em todas as interpolações dinâmicas dos templates e manter regressões para payloads de markup.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 19 ---

ISSUE 20 (F-21)

--- ISSUE 20 ---

Titulo: [Seguranca] [ALTA] EmailRenderService usa escaping contextual no HTML

Labels: security, alta

Descricao

A função de escaping deixou de ser código morto: o renderer agora aplica o tratamento adequado ao contexto de saída.

Evidencia

Arquivo: `backendnode/src/api/email-templates/email-render.service.ts:13-26; 100-124`

interpolate recebe escapeValues; render chama interpolate(..., true) para HTML e false para texto/assunto. Valores ausentes p

Impacto

Templates persistidos não inserem dados de usuário diretamente no HTML sem escaping; texto e assunto não recebem escaping HTML

Sugestao de correcao

Refatorar a interpolação para distinguir contexto HTML de texto, validar URLs dinâmicas antes do escaping e cobrir campos descon

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 20 ---

ISSUE 21 (F-22)

--- ISSUE 21 ---

Titulo: [Seguranca] [ALTA] EmailService escapa texto simples antes de convertê-lo em HTML

Labels: security, alta

Descricao

A interpolação direta de text em um parágrafo HTML permitia que callers transformassem markup em conteúdo executável.

Evidencia

Arquivo: `backendnode/src/email/email.service.ts:279-284`

No caso simple, o serviço usa `

\${escapeHtml(text).replace(/\n/g, '
')}

` e mantém o conteúdo original apenas no camp

Impacto

Payloads HTML enviados como texto simples são serializados como texto no HTML do email.

Sugestao de correcao

Aplicar escapeHtml ao texto antes de inserir a marcação de quebra de linha e cobrir script/markup nos testes de EmailService.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 21 ---

ISSUE 22 (F-23)

--- ISSUE 22 ---

Titulo: [Seguranca] [MEDIA] Previews HTML de templates usam DOMPurify

Labels: security, media

Descricao

Os dois sinks frontend que renderizam HTML bruto foram tratados; não há confiança direta no HTML vindo do renderer de templates

Evidencia

Arquivo: `frontend/app/lib/sanitize-html.ts; frontend/app/components/admin/email-templates/email-template-preview-modal.tsx; em

sanitizeHtmlPreview usa DOMPurify.sanitize com configuração explícita de tags/atributos permitidos. O helper é aplicado tanto

Impacto

Markup, scripts, event handlers e protocolos perigosos são removidos antes da renderização no painel administrativo.

Sugestao de correcao

Adicionar DOMPurify fixado, centralizar a sanitização em helper reutilizável e cobrir os dois sinks com regressões de XSS e mar

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado

- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 22 ---

ISSUE 23 (F-24)

--- ISSUE 23 ---

Titulo: [Seguranca] [BAIXA] termo-adesao-text.ts escapa placeholders dinâmicos

Labels: security, baixa

Descricao

A função aceitava dados futuros do usuário e os inseria sem proteção contextual, criando um vetor adormecido em documento legal

Evidencia

Arquivo: `frontend/app/lib/termo-adesao-text.ts:1-45`

fill aplica escapeHtml aos campos founderName, founderCpf, startupName, startupCnpj e companyEmail antes de substituir placeh

Impacto

Valores dinâmicos com markup são tratados como texto no termo; a substituição futura não reabre XSS por interpolação direta.

Sugestao de correcao

Escapar cada valor dinâmico no contexto HTML antes da substituição e cobrir scripts, atributos e placeholders nos testes unitár

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 23 ---

ISSUE 24 (F-25)

--- ISSUE 24 ---

Titulo: [Seguranca] [BAIXA] Sanitização padronizada entre transparency e email-templates

Labels: security, baixa

Descricao

A inconsistência histórica era a ausência de sanitização nos previews HTML, não uma necessidade de substituir o pipeline Markdow

Evidencia

Arquivo: `frontend/app/components/transparency/*; frontend/app/lib/sanitize-html.ts; email-template-preview-modal.tsx; email-ter

Transparency mantém rehype-sanitize para Markdown; os sinks HTML bruto de email-templates usam sanitizeHtmlPreview baseado em

Impacto

Conteúdo Markdown e HTML bruto não são renderizados sem a camada de sanitização correspondente.

Sugestao de correcao

Preservar rehype-sanitize no Markdown e aplicar DOMPurify centralizado em todos os previews HTML bruto.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 24 ---

ISSUE 25 (F-26)

--- ISSUE 25 ---

Titulo: [Seguranca] [INFORMATIVA] Transparência usa rehype-sanitize para conteúdo Markdown

Labels: security, informativa

Descricao

Este achado é um controle positivo da auditoria e permanece válido após a adição do sanitizer específico para previews HTML brutos.

Evidencia

Arquivo: `frontend/app/components/transparency/\*:varia`

discussion-replies, post-detail e transparency-featured-report importam rehype-sanitize e o utilizam no pipeline ReactMarkdown.

Impacto

Conteúdo Markdown de posts e replies permanece mitigado contra markup e scripts não permitidos.

Sugestao de correcao

Nenhuma alteração necessária; manter rehype-sanitize e sua cobertura de uso nos componentes de transparência.

Criterios de aceite

- [] Teste de regressao que prova o bug existe antes do fix (RED)
- [] Fix aplicado
- [] Teste de regressao passa (GREEN)
- [] Auditoria manual confirma que o vetor esta fechado
- [] PR revisado por outro dev
- [] Documentacao (AGENTS.md ou CASE.md) atualizada com a regra

--- FIM ISSUE 25 ---